



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing

Class: BSCS-2023

Section:

Course: CS-346 Compiler Construction

Lab 02: Lab Topic

Date:

Lab Instructor: Ms. Urooj Akmal

Student Name: Ahmed Hussain

Lms Id: 547096



Introduction

ARRAYS

Objectives

Pointers and arrays

Tools/Software Requirements

Visual studio code

Lab Tasks

Task 1:

What p means: p is a local pointer variable holding the memory address of the very first element of the built-in array (&sales[0]).

What &p means: &p refers to the memory address of the pointer variable p itself as it resides on the stack, which is distinct from the address *stored* inside p.

What *p means: This is the dereferencing operator. It retrieves or modifies the actual integer value stored at the memory location pointed to by p (i.e., sales[0]).

What *(p + 2) means: Using pointer arithmetic, p + 2 shifts the address forward by 2 integer positions, pointing directly to sales[2]. Dereferencing it (*(p + 2)) accesses the value of the third day's sales.

Valid indices: For an array declared as int sales[5];, the valid indices are strictly 0, 1, 2, 3, and 4.



Why $*(p + 5)$ cannot be used: Index 5 is completely out of bounds. Because the array only allocated space for 5 elements (indices 0 to 4), trying to read or write via $*(p + 5)$ accesses unauthorized neighboring memory, resulting in **undefined behavior** or program instability.

```
/*
Task 1: Shop item sales over five days using fixed-size array and pointers
*/
#include <iostream>
using namespace std;

int main() {
    int sales[5];
    int* p = sales;

    cout << "Enter sales for 5 days:\n";
    for (int i = 0; i < 5; ++i) {
        cin >> *(p + i);
    }

    int total = 0;
    cout << "Sales values: ";
    for (int i = 0; i < 5; ++i) {
        cout << *(p + i) << " ";
        total += *(p + i);
    }
    cout << "\nTotal sales: " << total << endl;

    *(p + 2) += 2;

    total = 0;
    cout << "Updated sales values: ";
    for (int i = 0; i < 5; ++i) {
        cout << *(p + i) << " ";
        total += *(p + i);
    }
    cout << "\nUpdated total sales: " << total << endl;

    return 0;
}
```

```
C:\WINDOWS\system32\cmd. x + v
Enter sales for 5 days:
5
9
8
7
4
Sales values: 5 9 8 7 4
Total sales: 33
Updated sales values: 5 9 10 7 4
Updated total sales: 35
Press any key to continue . . . |
```

Task 2:

Part(3&4)

Why the pointer variable and the allocated elements are different objects (Part 3): The pointer variable (marks) is a stack-allocated variable that merely holds a memory address. The elements created by `new int[n]` are an independent block of memory dynamically allocated on the **heap**. Deleting the array using `delete[] marks;` destroys the heap block, but the pointer variable `marks` itself stays on the stack until it goes out of scope (which is why we explicitly set `marks = nullptr;` to prevent a dangling pointer).

Why entering `n` at runtime works with `new int[n]` vs `int marks[n];` (Part 4): `new int[n]` requests a block of memory from the **heap** at runtime, where sizes do not need to be known until execution. Conversely, `int marks[n];` attempts to allocate a fixed-size stack array. Standard C++ requires stack-allocated array sizes to be compile-time constants because stack frame layouts must be completely determined during compilation.



```
Source.cpp
(Global Scope)
// Task 2: Dynamic 1D array for student marks with runtime size allocation
//
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter number of students (1-10): ";
    cin >> n;

    if (n < 1 || n > 10) {
        cout << "Error: no allocation or mark input" << endl;
        return 1;
    }

    int* marks = new int[n];
    cout << "Enter " << n << " marks (0 to 100): ";
    for (int i = 0; i < n; ++i) {
        cin >> *(marks + i);
    }

    int total = 0;
    int passCount = 0;
    cout << "Marks entered: ";
    for (int i = 0; i < n; ++i) {
        int current = *(marks + i);
        cout << current << " ";
        total += current;
        if (current >= 50) {
            passCount++;
        }
    }
    cout << endl;

    double average = static_cast<double>(total) / n;
    cout << "Total: " << total << endl;
    cout << "Average: " << fixed << setprecision(2) << average << endl;
    cout << "Pass count (>=50): " << passCount << endl;

    delete[] marks;
    marks = nullptr;

    return 0;
}
```

```
C:\WINDOWS\system32\cmd. X
Enter number of students (1-10): 2
Enter 2 marks (0 to 100): 70
74
Marks entered: 70 74
Total: 144
Average: 72.00
Pass count (>=50): 2
Press any key to continue . . . |
```

Task 3 Part(3)

What rowPtr + 1 points to and why its step differs from an int*:

rowPtr is defined as `int (*rowPtr)[3]`, meaning it is a pointer to an *entire array of 3 integers*. Therefore, adding 1 to rowPtr steps over an entire row (i.e., $3 \times \text{sizeof(int)}$ bytes) to point to the start of the second row. In contrast, a standard `int*` pointer only steps over a single integer (sizeof(int) bytes).

Why `int** p = sales`; should not be used:

A built-in 2D array (`int sales[2][3]`) is laid out in memory as a single, large, contiguous block of sequential bytes. A double pointer (`int**`) expects an array of independent row pointers pointing to scattered heap locations. Assigning a built-in 2D array to an `int**` causes a type mismatch and compilation failure (or unsafe type casting).



The screenshot shows a Visual Studio IDE with a C++ project named 'ConsoleApplication2'. The source file 'Source.cpp' contains the following code:

```
// Task 3: Fixed-size 2D array representing branches and days using row pointers
//
#include <iostream>
using namespace std;

int main() {
    int sales[2][3];
    int *rowPtr[3] = sales;

    cout << "Enter sales for Branch 1 (3 days) then Branch 2 (3 days):\n";
    for (int r = 0; r < 2; ++r) {
        for (int c = 0; c < 3; ++c) {
            cin >> *(rowPtr + r) + c;
        }
    }

    cout << "\nSales Table (Rows: Branches, Cols: Days):\n";
    for (int r = 0; r < 2; ++r) {
        cout << "Branch " << (r + 1) << ": ";
        for (int c = 0; c < 3; ++c) {
            cout << *(rowPtr + r) + c << " ";
        }
        cout << endl;
    }

    cout << "\nBranch totals:\n";
    for (int r = 0; r < 2; ++r) {
        int branchTotal = 0;
        for (int c = 0; c < 3; ++c) {
            branchTotal += *(rowPtr + r) + c;
        }
        cout << "Branch " << (r + 1) << " total: " << branchTotal << endl;
    }

    cout << "\nDay totals:\n";
    for (int c = 0; c < 3; ++c) {
        int dayTotal = 0;
        for (int r = 0; r < 2; ++r) {
            dayTotal += *(rowPtr + r) + c;
        }
        cout << "Day " << (c + 1) << " total: " << dayTotal << endl;
    }

    return 0;
}
```

The output window shows the following execution results:

```
Enter sales for Branch 1 (3 days) then Branch 2 (3 days):
2
1
8
7
5
7

Sales Table (Rows: Branches, Cols: Days):
Branch 1: 2 1 8
Branch 2: 7 5 7

Branch totals:
Branch 1 total: 11
Branch 2 total: 19

Day totals:
Day 1 total: 9
Day 2 total: 6
Day 3 total: 15
Press any key to continue . . .
```

The output window also shows the build process:

```
1>----- Build started: Project: ConsoleApplication2, Configuration: Debug Win32 -----
1> Source.cpp
1> ConsoleApplication2.vcxproj -> C:\Users\ahussain.bsccs25seecs\Documents\Visual Studio 2013\Projects\ConsoleApplication2\Debug\ConsoleApplication2.exe
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
```

Task 4:

Task 4 Explanations (Part 5)

- **What would remain allocated if only `delete[]` marks; were executed:** If only `delete[]` marks; were called, the outer array holding the row pointers (`int**`) would be destroyed, but the dynamically allocated individual row arrays (`marks[0]`, `marks[1]`, etc.) would remain stranded in heap memory. This results in an orphaned memory block and a severe **memory leak**.



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
Source.cpp (Global Scope)
int** marks = new int*[rows];
for (int r = 0; r < rows; ++r) {
    marks[r] = new int[cols];
}

cout << "Enter marks for each student row by row:\n";
for (int r = 0; r < rows; ++r) {
    for (int c = 0; c < cols; ++c) {
        cin >> *(marks + r) + c;
    }
}

cout << "\nMarks Matrix:\n";
for (int r = 0; r < rows; ++r) {
    for (int c = 0; c < cols; ++c) {
        cout << *(marks + r) + c << " ";
    }
    cout << endl;
}

int bestStudent = 1;
int maxTotal = -1;

for (int r = 0; r < rows; ++r) {
    int currentTotal = 0;
    for (int c = 0; c < cols; ++c) {
        currentTotal += *(marks + r) + c;
    }
    cout << "Student " << (r + 1) << " Total: " << currentTotal << endl;

    if (r == 0 || currentTotal > maxTotal) {
        maxTotal = currentTotal;
        bestStudent = r + 1;
    }
}

cout << "Top student: " << bestStudent << " (Total: " << maxTotal << ")" << endl;

for (int r = 0; r < rows; ++r) {
    delete[] marks[r];
}
delete[] marks;
marks = nullptr;

return 0;
}

Output
Show output from: Build
1>----- Build started: Project: ConsoleApplication2, Configuration: Debug Win32 -----
1> Source.cpp
1> ConsoleApplication2.vcxproj -> C:\Users\ahussain.bscecs25seecs\Documents\Visual Studio 2013\Projects\ConsoleApplication2\Debug\ConsoleApplication2.exe
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
```

Task 5:

```
Source.cpp (Global Scope)
//Task 5: Dynamic array expansion by copying elements to accommodate a new student mark
//
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n (1-10): ";
    cin >> n;
    if (n < 1 || n > 10) {
        cout << "Invalid size." << endl;
        return 1;
    }

    int* marks = new int[n];
    cout << "Enter " << n << " marks: ";
    for (int i = 0; i < n; ++i) {
        cin >> *(marks + i);
    }

    int newMark;
    cout << "Enter new mark to append: ";
    cin >> newMark;

    int* newMarks = new int[n + 1];
    for (int i = 0; i < n; ++i) {
        *(newMarks + i) = *(marks + i);
    }
    *(newMarks + n) = newMark;

    delete[] marks;
    marks = newMarks;
    n += 1;

    cout << "Updated marks: ";
    for (int i = 0; i < n; ++i) {
        cout << *(marks + i) << " ";
    }
    cout << endl;

    delete[] marks;
    marks = nullptr;

    return 0;
}
```



Task 5 Explanations (Part 3)

- **Why changing n alone does not increase the old block's capacity:** The variable n is merely a local integer tracking count residing on the stack. Changing n (e.g., $n = n + 1$) does **not** send any command to the operating system to expand the physical heap chunk allocated by `new int[n]`. The OS allocates a rigid, fixed size of memory. Attempting to store data beyond index $n-1$ without explicit reallocation causes out-of-bounds corruption.

Task 6:

The screenshot shows a Visual Studio Code editor with a C++ file named `Source.cpp`. The code is as follows:

```
#include <iostream>
using namespace std;

int main() {
    int n = 3;
    int* values = new int[n];

    cout << "Enter 3 integers (e.g., 4 5 6): ";
    for (int i = 0; i < n; i++) {
        cin >> *(values + i);
    }

    cout << "Stored values: ";
    for (int i = 0; i < n; i++) {
        cout << *(values + i) << " ";
    }
    cout << endl;

    delete[] values;
    values = nullptr;

    return 0;
}
```

The output window shows the following text:

```
Enter 3 integers (e.g., 4 5 6): 4
5
6
Stored values: 4 5 6
Press any key to continue . . . |
```

Task 6 Explanations (Part 1)

1. **Out-of-Bounds Access (Buffer Overflow):**
 - *Problem:* The loop condition `for (int i = 0; i <= n; i++)` runs for $i = 0, 1, 2, 3$ when $n = 3$.
 - *Consequence:* It attempts to access `values[3]`, which is outside the allocated bounds (valid indices are 0, 1, 2). This leads to undefined behavior or overwriting adjacent memory.
2. **Incorrect Deallocation:**
 - *Problem:* The code uses `delete values;` instead of `delete[] values;`.
 - *Consequence:* Because the memory was allocated as a dynamic array (`new int[n]`), using plain `delete` instead of `delete[]` causes undefined behavior and potential memory corruption.
3. **Use After Free (Dangling Pointer):**
 - *Problem:* The code executes `cout << values[0];` *after* the memory has already been deleted.



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

- *Consequence:* It reads from a dangling pointer (a memory address that is no longer valid), leading to unpredictable results or crashes.

Github link: <https://github.com/ahmedimcb12-bot/lab01-dsa/commits/main/0e15753625c829825a0485a209a142235cc1bee8>